

4

진도 (공식 교재 기준)

- 2-1_자연어 처리 기본
 - 사전 학습 기반 언어 모델 (BERT, T5, GPT-1/2)

강의자료

- 자연어처리
 - 01 Introduction to Text Analytics.pdf
 - 02 전처리.pdf
 - 03 2024- Text Vectorization - Part1.pdf
 - 04 2024- Text Vectorization - Part2.pdf
 - 2-1_자연어 처리 기본 (싸피 자료)

강의 주제

오늘 배울 내용은 대부분이 싸피에서 다루지 않은 내용임
그러나 실제로 자연어처리를 할 때는 아래의 개념들을 숙지하는게 좋음
이제부터 자연어처리 (전공자 버전) 을 배워볼거임

자연어처리 소개

→ 01 Introduction to Text Analytics.pdf

17P. 자연어처리에서는 어떤거함?

자연어처리에서는 텍스트를 처리해서 가치 있는 정보를 추출해볼거임

42P. 예시가 어떤 것들이 있음?

스팸과 일반 메일 분류, (46~48) 문장 감정 분석, (52) 감성 분석 기반 추천, (53)수요예측, (56)문서 요약, (59) 번역

할 수 있는 일의 종류는 거의 무한하다 할 수 있음

67P. 자연어처리의 주요 난관

1. 단어의 개수가 너무 많음: 한국어는 표준국어대사전 기준 50만개 이상의 단어가 있음. 한 문장을 쓸 때도 기계는 이 50만개의 단어 중 다음 단어를 선택해야 함
2. 자연어는 너무 모호(Ambiguous)함: 동음이의어 같은거 생각하면 될듯
3. 자연어는 정제되지 않은 형태: csv나 json같은 형태로 정리된 형태가 아닌 날 것 그대로의 상태임 → 컴퓨터가 처리하려면 추가적인 조정이 필요함

71P. NLP의 주요 프로세스

1. Raw Text 수집: 크롤링
2. Preprocessing (전처리): 토큰화, 불용어 처리 등
3. Vectorization (Embedding)
4. Model 구축
5. Prediction (Inference, 추론)

Preprocessing (전처리)

→ 02 전처리.pdf

3P. 주요 용어 정리

Corpus: 말뭉치, 말 그대로 우리가 수집한 텍스트 데이터 전체

Token: 문장을 쪼갤 때의 가장 작은 단위

형태소: 의미를 잃지 않으면서 나타낼 수 있는 가장 작은 단위 (한국어 문법)

4P. Tokenization

사전에 설정한 Token이라는 단위로 Corpus를 나눠봅시다

최대한 잘게 자르는게 좋지만, 그렇다고 문자 단위로 분해해버리면 학습 시 비효율성이 커질거 같음

(그리고 어떠한 단어가 들어가는지, 전체적으로 문장이 어떤 뜻인지 알기가 굉장히 뻥셀거같음)

(9) 그래서 주로 형태소로 자르거나(🇰🇷, 🇯🇵, 🇨🇳), 띄어쓰기로 자름(🇬🇧, 🇪🇸, 🇫🇷)

띄어쓰기로 잘라도 되는 언어는 별 문제 없는데, 한국어는 띄어쓰기로 하면 이상해짐

"사과가 맛있다.", "사과는 맛있어" ← 이거 같은 의미 문장인듯. 그런데 띄어쓰기로 나눠보면

사과가 ↔ 사과는, 맛있다 ↔ 맛있어 ← 토큰이 다르니 컴퓨터는 아예 다른 의미를 가지고 있구나 하고 읽음

그래서 한국어는 주로 형태소로 자름

"사과가 맛있다" → "사과" "-가" "맛있" "-다"

이렇게 4개로 잘라서 자연어처리를 시작할거임

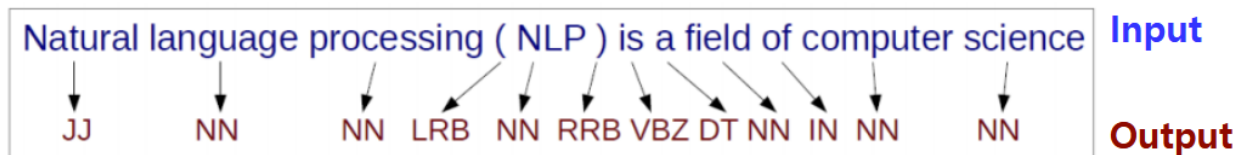
19P. POS Tagging

이렇게 토큰으로 잘랐으면 여기에 품사 정보를 붙임

동음이의어 걸러내기 쉽게 하려고 이런거 하는데 다만 이건 전통적인 방식이고 요즘 LLM들은 이런거 안하고 문맥에서 문법적 정보를 내부적으로 학습해버리는 경우가 많음

"I read a book", "I book a hotel" 여기서 둘다 book 이라는 단어가 있지만, 하나는 "책(명사)"이라는 의미로 쓰였고 하나는 "예약하다(동사)"라는 의미로 쓰임.

(27) 그래서 형태소 단위로 문장을 분해한 다음 각 토큰에 품사 태깅(POS)을 해줌



JJ, NN 이런건 각 품사의 코드임. Tokenizer 안에 정의되어 있음

이 문장을 RNN에 넣던가 해서 각 토큰 별 품사를 뽑아내고, 그 정보를 토큰에 기록해놓음

34P. 불용어(Stopword)처리

사실 모든 토큰을 사용하는건 아님

유의미한 토큰만을 선별하는 작업을 불용어 처리라 하는데, 조사, 접미사 같은 토큰들은 없어도 문장 의미 해석에 별 지장이 없기 때문에 이런 토큰들은 아예 날려버림

사용자가 직접 불용어 사전을 정의하는 경우가 많음

"나는 오늘 학교에서 친구와 함께 맛있는 점심을 먹었다." 를 형태소 단위로 토큰화하면

나 / 는 / 오늘 / 학교 / 에서 / 친구 / 와 / 함께 / 맛있 / 는 / 점심 / 을 / 먹 / 었 / 다

여기서 는, 에서, 와 이런건 없어도 누가 뭘 했는지 파악하는데 크게 지장은 없을듯

그래서 불용어를 제거하면 아래와 같음

나 / 오늘 / 학교 / 친구 / 함께 / 맛있 / 점심 / 먹

다만 불용어를 제거할때는 잘 생각해보고 제거해야 함

⇒ 트랜스포머의 시대가 오며 개별 토큰과 문장 전체의 문맥을 비교할 수 있게 되어 이제는 명시적으로 잘 하지 않는 작업

현대의 전처리

현대에는 어텐션과 트랜스포머로 인해 개별 토큰과 문장 전체를 직접 비교할 수 있게 됨.

그래서 형태소 토큰화 → POS → 불용어처리와 같은 전통적인 파이프라인은 거의 쓰지 않음 (Transformer 기반의 모델로 NLP를 한다면)

대신 서브워드 토큰화 → 트랜스포머 구조가 많이 쓰임

서브워드 토큰화는 기본적으로는 형태소처럼 글자 단위보다는 더 크게 자름
핵심 아이디어는 자주 등장하는 문자열은 크게 묶고, 드문 단어는 더 잘게 쪼개는 방식임

GPT가 쓰는 BPE(Byte Pair Encoding)는 이렇게 학습됨

1. 일단 바이트 단위로 아주 잘게 쪼개봄
2. 그런 다음 데이터 전체를 분석해봄
3. 분석 결과 자주 붙어 나오는 조각들은 합침
4. 이거 반복

자연어처리 → 자 / 연 / 어 / 처 / 리 → 데이터를 보니 보통 자+연 이라고 쓰네? → 자연 / 어 / 처 / 리 → 자연+어 도 자주 붙여서 쓰네? → 자연어 / 처 / 리 → 처+리도 자주 붙이는듯 → 자연어 / 처리 → 토큰화 완료

텍스트 벡터화

→ 03 2024- Text Vectorization - Part1.pdf

(6) 자연어를 토큰 단위로 쪼갬으면 이제 이걸 기계에 집어넣어야 함
근데 CPU는 글자가 뭔지 모름 (0, 1 밖에 모름)

(9) 1번 아이디어: 문장의 각 토큰에 인덱스를 추가해서 일대일대응으로 하면 되지 않을까요?

사과 → 1, 자동차 → 2, 드라이버 → 3

→ 근데 이렇게 하면 단어 간 유사도 어떻게 구함

사과 라는 단어는 드라이버 라는 단어보다 자동차 라는 단어에 더 가깝나? ← 뭔가 이상한듯

(10) 2번 아이디어: 원핫 인코딩 쓰면 되지 않을까요?

	사과	자동차	드라이버
사과	1	0	0
자동차	0	1	0
드라이버	0	0	1

이렇게 하면 텍스트가 일종의 벡터가 됨 (사과 = [1, 0, 0])

이제 수학적으로 유사도(코사인 유사도) 계산이 가능해짐

근데 문제가 있음

1. 토큰이 몇천만개면 벡터의 크기도 몇천만 차원으로 커지는데다가, 그 수많은 0 중 단 하나의 값만 1임 → 너무 비효율적인듯
2. 유사도를 계산할 순 있지만, 사과 라는 단어는 드라이버 라는 단어보다 자동차 라는 단어에 더 가깝다는 문제는 해결되지 않음

Dense Embedding

그래서 요즘은 원핫벡터 방식 안씀

모델 안에 모든 단어들에 대한 벡터들을 저장해놓고, 필요할때마다 꺼내쓰는 방식을 사용

사과 = [0.8, 0.2, 0.7, ...]

자동차 = [0.7, 0.3, 0.6, ...]

드라이버 = [-0.2, 0.9, 0.1, ...]

이 벡터들의 값은 모델이 학습하면서 스스로 정함 (의미가 비슷한 단어이면 비슷한 숫자가 되도록 함)

학습 전에는 랜덤한 숫자들로 채워놨다가, 학습을 시작하면서 엄청난 양의 문장에 대해 예측을 잘하도록 이 벡터값들을 모두 수정함

14P. Bag of Words

아까 Dense Embedding은 개별 토큰에 대한 이야기였음

우리는 이제 문장 전체를 숫자로 표현해보고 싶음

문장 A: 나는 사과를 먹었다

문장 B: 나는 바나나를 먹었다

문장 C: 자동차가 빠르게 달렸다

이런 문장이 있을 때 A와 B는 비슷하고, C는 좀 다르다 라고 유사도를 계산하고 싶은거임
그러기 위해서는 문장 전체를 하나의 벡터로 표현할 방법이 필요함

가장 단순한 아이디어: 각각 단어 몇 번 나왔는지 세기

1. 먼저 전체 문서에서 등장하는 각각의 단어 목록을 만듦

[나는, 사과를, 먹었다, 바나나를, 자동차가, 빠르게, 달렸다] → 이것 Vocabulary라 함

2. 그 다음 각 문장에서 단어가 등장한 횟수를 세봄

문장 A → [1, 1, 1, 0, 0, 0, 0]

문장 B → [1, 0, 1, 1, 0, 0, 0]

문장 C → [0, 0, 0, 0, 1, 1, 1]

그렇게 하면 문장을 하나의 벡터로 표현 가능할 듯 하다

⇒ 이게 바로 Bag of Words, BoW임

말 그대로 문장 전체를 각 단어가 담겨져있는 가방으로 보겠다 이거임

하지만 문제가 있음

문법이나 어순을 무시하기 때문에 (단어가 등장한 빈도만 보기 때문에)

나는 너를 좋아한다

너는 나를 좋아한다

이 두 문장은 BoW의 관점에서는 같은 문장임 (같은 단어가 같은 개수만큼 들어있으니)

또한 문장이 길어지면 (문장 속의 단어 종류가 많아지면, Vocabulary가 커지면) 어떡할거임
벡터값이 엄청 커지는데, 모든 단어가 들어가는 문장은 별로 없을테니 각 문장 벡터들은 대부분 0으로 채워져 있을거임

한가지 더 있음

단순히 빈도만 보는 특성상 영어의 "The"나 한국어의 "그리고" 처럼 문법 상 반복되는 단어들

이 매우 중요하게 취급됨

당연히 the는 중요한 정보가 아니니 이걸 걸러내는 방법이 또 필요할거같음

21P. TF-IDF

BoW의 가장 치명적인 문제는 the 같은 많이 등장할 수 밖에 없는 단어들이 중요하게 취급된단 거임

그런데 실제로 그러한가? → 아마 아닌 느낌임

⇒ “많이 등장하는 단어 ≠ 이 문서를 잘 설명하는 중요한 단어” 라는 사실을 알게 됨

실제로 뉴스 기사를 생각해보면 the는 거의 모든 기사에서 나올거임

반면, (다른 기사에서는 별로 안나오는데) 특정 기사에서 semiconductor라는 단어가 계속 반복된다면 “아, 이 기사는 반도체와 관련된 기사구나” 라고 판단해도 될거같음

⇒ 이게 TF-IDF의 간략한 설명임

TF-IDF는 TF와 IDF 이 2개로 이루어져 있음

TF: Term Frequency, 이 문서 속에 이 단어가 몇 번이나 나왔는가? 에 대한 값임

IDF: Inverse Document Frequency, 이 단어가 몇 개의 문서에서 나왔는가? 에 대한 값임

IDF ← 여기 왜 Inverse가 붙음?

먼저 TF는 BoW와 같은 방식으로 계산함

각 문장에 대해 각 Vocabulary가 몇 번이나 들어있는지 계산하는거임

다음으로 IDF를 계산하기 앞서 DF부터 계산할거임

DF는 특정 단어가 몇 개의 문장에서 나왔는지를 세면 됨

그리고 그걸 역수로 해서 로그를 취함

$$IDF = \log\left(\frac{N}{DF}\right)$$

N은 전체 문장의 개수, DF는 우리가 방금 구한 값임

그래서 이게 왜 Inverse지?

⇒ 문장이 100개라고 해 보자

“그리고” 라는 단어가 100개의 문장에서 모두 등장했다고 가정하면 DF 값은 100일거임

그리고 그거의 IDF를 구해보면 $\log(100/100) = \log(1) = 0$ 이 나옴

즉, “그리고” 라는 단어는 IDF 값이 0이니 전혀 중요한 단어가 아니다 라고 판단하기 용이함

이번엔 “컴퓨터”라는 단어가 1개의 문장에서만 등장하면 DF의 값은 1임

IDF는 $\log(100/1) = \log(100)$ 이 됨 (로그의 밑은 정해져있지 않음. 어차피 상대적인 순서가 중요하기 때문에 밑은 아무거나 써도 됨)

$\log(100) > 0$ 이기 때문에 "컴퓨터" 라는 단어는 "그리고" 라는 단어보다 훨씬 중요함

근데 왜 하필 log를 씀?

문장의 개수(N)가 커질수록 IDF가 기하급수적으로 너무 커지기 때문에 log를 씀

여기까지 잠깐 정리

BoW는 단어들의 등장 회수를 썼지만, 여러 문제가 있었음

그걸 보완하기 위해 TF-IDF가 나왔음

하지만 두 방법 모두 공통된 문제가 있는데, "자동차" 와 "승용차" 가 비슷한 단어라는 사실을 표현하지 못한단 거임

⇒ 그렇다면 단어가 가진 의미 그 자체를 숫자(벡터)로 표현할 수는 없을까?

텍스트 벡터화: Word2Vec

→ 04 2024- Text Vectorization - Part2.pdf

당연히 가능하다

단어 자체를 벡터로 바꾼다고 이름이 Word2Vec임

우선 일반적인 문장을 생각해보자

비슷한 문맥에서 등장하는 단어는 비슷한 의미를 가지지 않을까?

나는 아침에 사과를 먹었다
나는 아침에 바나나를 먹었다
나는 아침에 딸기를 먹었다
사과는 맛있는 과일이다
바나나는 맛있는 과일이다
딸기는 맛있는 과일이다

이런 문장들이 Corpus에 가득 들어있다면, 이걸 학습하는 컴퓨터도 사과, 바나나, 딸기가 비슷한 주변 단어들과 함께 등장한다는 걸 발견할 수 있을 것임

실제로 사과 자리에 바나나를 넣고, 딸기 자리에 사과를 넣어도 전혀 위화감이 없음

따라서 사과, 바나나, 딸기라는 단어의 벡터들은 아무래도 비슷한 것이 틀림없는데, 이걸 숫자로 어떻게 표현하면 될까?

33P. CBOW

우선, 일반적인 문장을 하나 준비하자

나는 맛있는 사과를 매일 먹는다

그리고 이 문장에서 단어 하나를 빈칸으로 뚫어버리는거임

나는 맛있는 [???] 매일 먹는다

그 다음 DNN한테 이 ???를 맞춰봐라 라는 문제를 낸다

DNN의 입력으로는 나머지 문장, 즉 나는, 맛있는, 매일, 먹는다 가 입력되고, 출력은 빈 칸에 들어갈 단어가 출력되게 하면 되지 않을까?

제대로 학습이 되었다면 "사과를" 을 출력할거 같음

근데 나는, 맛있는, 매일, 먹는다 를 뉴럴넷에 어떻게 넣지?

그걸 위해 아까 봤던 Dense Embedding의 개념을 잠시 떠올려보자

사과 = [0.8, 0.2, 0.7, ...]

자동차 = [0.7, 0.3, 0.6, ...]

드라이버 = [-0.2, 0.9, 0.1, ...] ← 이게 바로 Dense Embedding 이었음

그럼 이와 비슷하게 나는, 맛있는, 매일, 먹는다 이 4가지 단어를 3개의 숫자를 써서 나타내면?

나는 → [0.2, -0.1, 0.7]

맛있는 → [0.8, 0.3, 0.1]

매일 → [0.1, 0.6, 0.4]

먹는다 → [0.4, -0.2, 0.9] ← 뭐 이런 식으로 표현되지 않을까

이럼 숫자이니 DNN의 입력으로 쓸 수 있을거같음

이렇게 벡터로 임베딩한 결과물을 Embedding Matrix라 부를거임

그리고 각 단어에 대응하는 벡터 (Embedding Matrix의 행)를 Dense Vector 라 부를거임

(35) 다시 문제를 떠올려 보자

나는 맛있는 [???] 매일 먹는다

우리는 이 ???에 들어갈 단어를 찾는게 목표였음

그럼 일단 ??? 주변에 있는 맛있는 과 매일 의 Dense Vector를 각각 DNN의 입력으로 넣은 뒤, Softmax를 취하면 모델이 알고 있는 모든 단어 (즉 Vocabulary)에 대해 다음 출력의 확률이 정해지지 않을까?

(강의자료는 Dense Embedding 대신 One-Hot Encoding을 사용하고 있음)

(42) 이 때 맛있는 과 매일 의 Dense Vector는 숫자가 총 6개 있음.

그럼 weight가 6 x (히든레이어 노드 개수) 만큼 필요할텐데, 사실은 맛있는과 매일 의 두 벡터를 하나로 압축해서 사용함

보통 평균을 많이 사용한다고 함

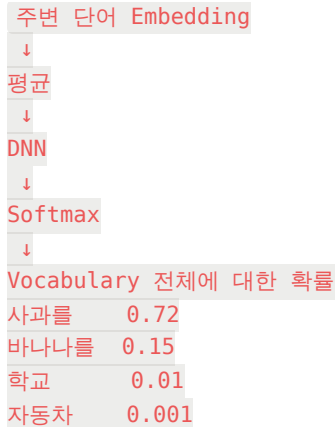
맛있는 → [0.8, 0.3, 0.1]

매일 → [0.1, 0.6, 0.4]

맛있는 + 매일 평균벡터 → [0.45, 0.45, 0.25] ← 이게 DNN 의 입력으로 들어가는 벡터임

나는 맛있는 [???] 매일 먹는다

↓



... ← DNN은 ???에 "사과를" 이 72%의 확률로 들어가야 한다고 계산함. (이 72%를 100%에 가깝게 만드는게 학습의 기본적인 목표임)

(46) 이 과정이 CBOW, Continuous Bag of Words임

왜 Continuous냐면 기존의 BoW나 One-Hot (둘다 정수, int로만 표현됨)과 달리 연속적인 실수 값(0.123 같은거, float)을 가질 수 있기 때문임

정리해보자면 CBOW는 문장 중간에 빈칸을 뚫고, 그 빈칸에 가장 적합한 단어가 뭔지? 문제를 푸는 방법임

그 방법으로 빈 칸의 앞뒤 단어를 보고 가장 그럴듯한 단어를 뽑는 알고리즘을 사용함

(만약 더 넓은 범위의 단어를 보고 싶다면 window size를 조절하면 됨. 1이면 바로 앞뒤만 보고, 2이면 앞 2개, 뒤 2개 단어를 봄. 그 후 4개 단어들의 Dense Vector들을 각각 DNN의 인풋으로 사용함)

49P. Skip-gram

CBOW는 문장에 빈 칸을 뚫어놓고 양 옆 단어를 이용해 빈칸에 뭐 들어갈지 추론하는 방법이었음

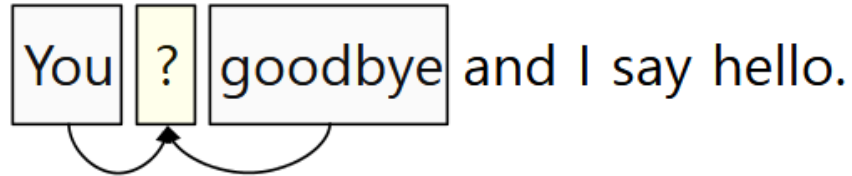
스킵그램은 반대로 단어 하나에 대해 양 옆에 빈칸을 뚫고, 그 단어를 이용해 양 옆 빈칸에 뭐가 들어갈지 추론하는 방법임

CBOW: You [??] goodbye and I say hello → ???에 뭐 들어갈지 You와 goodbye를 사용해 추론

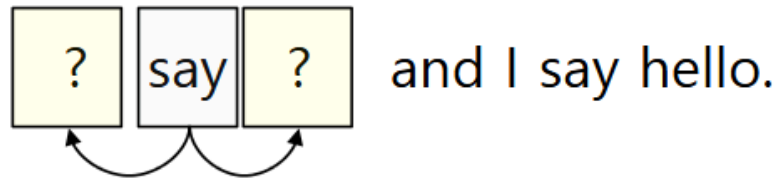
Skip-gram: [??] say [??] and I say hello → ???에 뭐 들어갈지 say를 이용해 2개를 동시에 추론

You say goodbye and I say hello.

CBOW:



Skip-Gram:



Skip-gram이 당연히 CBOW보다 평균적으로 더 어려운 대신에 CBOW는 연산이 더 빠르고 빈도가 높은 단어에 강하고, Skip-gram은 상대적으로 적은 데이터나 희귀 단어 표현에 장점이 있다고 함

Dense Embedding의 장점

이전의 One-Hot 같은 경우 Vocabulary의 크기만큼 0의 개수를(벡터의 길이를) 늘려야 함 만약 Vocabulary가 50만개라면,

사과 = [0, 0, 0, 0, 1, 0, 0,]
← 500,000차원 ← 0이 499,999개, 1이 1개임

반면 Dense Embedding은 벡터의 차원을 우리가 정할 수 있음 (하이퍼파라미터임)

⇒ Not High Dimensional

위에서 One-Hot으로 나타낸 사과의 벡터는 0이 50만개 가까이 있음 (전체의 99.99%가 0임)

보통 0은 값이 없다고 보기 때문에 이 벡터는 값이 굉장히 희소한(Sparse한) 벡터임

반면, Dense Embedding은 모든 차원에 숫자가 들어가있음

⇒ Not Sparseness

Dense Embedding은 벡터이기 때문에 유사도(코사인 유사도)를 계산할 수 있음.

즉, "어떤 두 단어가 얼마나 비슷한지" 를 계산할 수 있고, 정량적인 어떤 값으로 출력할 수 있음

⇒ Preserving Similarity between Words

Encoder 기반 모델: BERT

→ 2-1_자연어 처리 기본 (싸피 강의자료. 따로 안봐도 되게끔 아래에 자세하게 써놓음)

- 여기서 인코더 기반 모델이란 뜻은 트랜스포머 구조 중 인코더만 있다는 소리임

사실 Word2Vec(CBOW같은거)에도 문제가 있다

배 위에서 배를 먹었더니 두 배로 배가 불렀다.

이런 문장이 있을 때 "배" 라는 녀석은 , , ,  의 4 가지 다른 의미로 사용됨
그러나 CBOW는 이를 고려하지 않고 "배" 라는 단어의 임베딩 하나만 만들어버림

→ 이거를 해결해보자! 하고 나온게 BERT 임 (BERT는 역시 또 **대 구 글**에서 만듦)

BERT = **B**idirectional **E**ncoder **R**epresentations from **T**ransformers

CBOW와 달리 BERT는 해당 개별 토큰에 대해 문장 전체에 Self-Attention을 수행함.

따라서 각 토큰을 처리할 때 왼쪽과 오른쪽 문맥을 모두 참고할 수 있는데, 이것 Bidirectional 이라고 함.

조금 더 알아보도록 하자

Masked LM(Language Model)

예시 문장을 준비해보자 → 나는 오늘 학교에서 친구를 만났다

일부 토큰을 Mask함 (구멍뚫는거임) → 나는 오늘 [MASK]에서 친구를 만났다

그리고 BERT에게 MASK에 뭐가 있었게? 라고 물어보는거임

보다시피 정답은 "학교" 이다

그래서 "학교"라는 답을 뺀 확률이 높아지는 방향으로 학습함

근데 CBOW랑 뭐가 다르냐면, CBOW는 "오늘"과 "에서"의 벡터를 단순히 평균내서 DNN 돌렸을거임

반면 BERT는 전체 문장에 대해 Self-Attention을 수행한다

그렇게 되면 MASK 부분에 들어갈 단어를 학습할 때 문장 전체와 상호작용 하며 전체적인 문맥을 벡터에 담아낼 수 있게 됨

⇒ 이게 기본적인 MLM임. 싸피 강의자료에서는 더 정확하게 설명하니 더 알아보시다

아까의 문장을 다시 사용해보자 → 나는 오늘 학교에서 친구를 만났다 (문장 안에 토큰 10개정도 있다 치자)

BERT는 입력된 토큰들 중 약 15%의 토큰을 랜덤하게 선정해서 맞추는 문제로 만든다

일단 랜덤하게 선정된 토큰이 "학교"라고 해보자

그런 다음 BERT에게 단순히 학교를 MASK하는 것만 보여주는게 아닌, 다른 것도 보여줌

BERT가 풀어야 하는 문제 ⇒

A. 80% 확률로: 나는 오늘 [MASK]에서 친구를 만났다 ← 이거 답 뭐게
B. 10% 확률로: 나는 오늘 학교에서 친구를 만났다 ← 변화 없이 그대로 넣음
C. 10% 확률로: 나는 오늘 짜장면에서 친구를 만났다 ← 선택된 토큰을 랜덤한 다른 토큰으로 바꿔 치기함
(BERT는 이 중 어떤 문제가 들어왔는지도 모름)

C 문제일때를 보자

BERT는 학교 → 짜장면 으로 치환되었다는 사실을 모름

그냥 어텐션 돌려보니 짜장면이 이 문맥에서 나올 확률이 매우 낮다고 나옴

그렇다면 들어온 입력이 C 유형의 문제였나봄. (어떤 유형의 문제인지 판별하는건 아님)

“여기 들어갈 말은 짜장면이 아니라 학교였다” 라고 판단할 수 있는거임

왜 이렇게 하나면 A 문제만 들어오면 MASK의 위치 자체가 너무 친절한 힌트가 되기 때문임

NSP (Next Sentence Prediction)

BERT는 두 문장을 입력받고, 이 두 문장이 실제로 이어지는, 문맥이 유지되는 문장인지를 학습함

문장 A: 나는 배가 고파다. 문장 B:그래서 식당에 가서 밥을 먹었다. 문장 C: 서울은 오늘도 덥다.

A와 B가 입력되었다면, A와 B가 이어진 문장처럼 보이므로 출력으로 IsNext를 출력함

반면, A와 C가 입력되었다면, 이 둘은 별로 상관이 없는거같으므로 출력으로 NotNext를 출력함

실제로는 특별한 토큰을 붙여서 [CLS] 철수는 배가 고파다 [SEP]그래서 식당에 가서 밥을 먹었다 [SEP]

이런 느낌으로 만든 후 BERT에 집어넣음

다만 후속 연구에서는 NSP가 꼭 필요한지에 대한 의문이 제기되기도 했고, 실제로 NSP를 제거하고도 좋은 성능을 낸 BERT 버전도 있었음

Downstream Task = 이걸로 뭐할건데?

BERT에게 나무위키를 모든 문서를 넣어서 학습을 시켰다고 하자

그러면 BERT는 그동안의 학습으로 자연어, 특히 한국어가 어떻게 동작하는지를 배운 상태임
(내부 어딘가에 자연어 관련 내용들의 정보가 저장되어있음)

근데 우리는 BERT로 MASK 맞추기가 하고 싶은게 아니라, 리뷰 감정 분석, 스팸메일 분류, 개체명 인식 이런걸 하고 싶음

그래서 이미 한국어를 잘 아는 BERT를 데려다가 내가 원하는 작업을 수행할 수 있게 조금 더 학습시킴

이걸 Fine-tuning이라 함

스팸메일 분류를 예시로 들면, 정상 메일과 스팸메일 데이터셋을 만들어서 이런 메일이면 스팸이라고 추가적으로 학습시키는거임

BERT의 한계

꽤나 괜찮아보이는 모델이지만 한계가 너무 명확했음

1. 인코더는 정보를 입력받는 놈임. 그래서 본질적으로 뭔가를 생성하는 모델로 쓰기 불편함 (디코더가 없기 때문)
즉, 글을 읽는 눈과 이해하는 뇌는 있는데 그걸 표현하는 입이 없는거임. 이해는 잘하는데 말을 못함
2. 학습은 MASK 맞추기로 했는데, 실제 자연어 세상에서는 MASK따위 없음
3. 내가 원하는 태스크가 생길 때마다 매번 모델을 Fine-tuning 해야됨 (귀찮음)

Encoder - Decoder 기반 모델: T5

T5 = Text-to-Text Transfer Transformer, 만든 곳은 역시 **대 구 글**

BERT는 인코더만 가진 구조였음

근데 우리는 자연어처리로 하고 싶은 일이 스팸메일 분류만 있는건 아닌데?

번역, 요약, 챗봇, 문법 교정같은 태스크도 하고 싶단 말임

이것들은 입력과 출력이 모두 문장이라는 특징이 있음

그럼 트랜스포머처럼 인코더 디코더를 모두 쓰면 되지 않을까?

→ 이게 T5의 출발

Span Corruption

역시 예시 문장을 준비해보자 ⇒ 나는 오늘 학교에서 친구와 맛있는 점심을 먹었다

여기서 BERT의 MLM과 유사하게 MASK로 가릴건데, MLM과 달리 개별 토큰을 가리는 것이 아니라 Span을 가릴거임

Span이 뭐임? ⇒ 강 연속된 토큰 덩어리라 생각하면 됨

나는 [mask_1] 친구와 [mask_2] 먹었다 ← 이런걸 입력으로 넣는거임

그럼 디코더의 출력은 mask_1은 "오늘 학교에서", mask_2는 "맛있는 점심을" 이 출력될 것이다

근데 BERT랑 뭐가 다른거임? 이라는 질문이 나올 때가 됨

T5는 한 번에 여러 토큰들을 생성함 → 이미 문장 읽고 문장 생성하기를 연습한 셈임

Downstream Task

BERT와 달리 이제 자연어의 출력이 쉬워지면서 리뷰 감정 분석, 번역, 요약 등의 태스크가 BERT보다 훨씬 쉬워짐

T5의 한계

이거 확실히 BERT보단 쓸만한 물건이었음

근데 T5도 Downstream Task 할때 Fine-tuning 해야 하는건 똑같았음

파인튜닝 안하는 방법 없나?

Decoder 기반 모델: GPT

- 여기서 디코더 기반 모델이란 뜻은 트랜스포머 구조 중 디코더만 있다는 소리임

GPT = Generative Pre-trained Transformer

애는 드디어 구글 말고 다른 곳에서 만듦. 무려 OpenAI임

우선 GPT-1 부터 봅시다

Autoregressive Language Model

GPT의 학습은 ALM을 통해 이루어짐

우선 Autoregressive가 뭔지부터 살펴보자

Auto + Regressive 인데, Auto는 자동이란 뜻도 있지만, 사실 자기 자신이란 뜻도 있음

그러니까 Autoregressive는 “자기 자신(앞에서 나온 값들)을 보고 미래 데이터 예측” 이란 뜻임

Autoregressive Language Model은 자기가 앞에서 만든 결과를 다시 입력의 일부로 사용하면서 다음 값을 계속 예측하는 방식임

→ 근데 이거 한번 생성할때마다 계속 지금까지의 결과 넣으면 되게 비효율적인거 아님?

아님. 다시 트랜스포머의 구조를 생각해보면 디코더의 어텐션 중에 Masked Multi-Head Attention이 있었음

	<SOS>	나는	학생	이다
<SOS>				
나는				
학생				
이다				

이런 구조인데다가 실제 코드에서는 토큰 처리 과정이 병렬적으로 처리됨

입력				
나는	오늘	학교에	갔다	
↓	↓	↓		
오늘	학교에	갔다	...	

Input:	나는	오늘	학교에
Target:	오늘	학교에	갔다

조금 더 정확한 그림

그리고 마스크를 생각해보면 각 토큰은 자기보다 미래 토큰을 못봄

	볼 수 있는 것
나는	나는
오늘	나는 오늘
학교에	나는 오늘 학교에
갔다	나는 오늘 학교에 갔다

GPT-1의 한계

GPT-1은 문장에서 다음 단어 맞추기만 집중적으로 학습한 놀임

근데 우리가 원하는 태스크는 그런게 아니라니까?

그래서 GPT-1 까지는 내가 원하는 태스크에 맞는 파인튜닝이 필요했음

GPT-3 의 ICL(In-Context Learning)

GPT-1 보다 모델의 크기를 크게 키우고 엄청난 양의 텍스트를 학습시켰더니 흥미롭게도 파인 튜닝을 하지 않고도 새로운 태스크를 수행할 수 있는 능력이 나타나게 됨

GPT-3의 프롬프트에 문맥을 어떻게 집어넣냐에 따라 GPT-3가 프롬프트를 바탕으로 즉석으로 문맥을 파악해 새로운 태스크 수행이 가능해짐 (실제로 역전파가 일어나서 weight가 업데이트되는 건 아님)

만약 프롬프트에 예시를 하난도 안 주면 Zero-shot

예제를 딱 하나만 보여주면 One-shot

예제를 여러개 주면 Few-shot이라 함

근데 이걸로는 어려운 복잡한 문제를 풀어야 할 때도 있음
그건 ICL로 잘 안되더라

Chain of Thought

A 는 사과를 10개 가지고 있습니다.

B 에게 4개를 주고,

C 에게 남은 사과의 절반을 주었습니다.

A 에게 남은 사과는 몇 개입니까? ← 이런 문제는 ICL로 잘 안됨

사람이 이런 문제 어떻게 푸냐면

1. 처음에 10개

2. $10 - 4 = 6$ 개

3. $6 / 2 = 3$, $6 - 3 = 3$ 개

따라서 3개 남음

⇒ 이렇게 풀지 않음? 중간중간 추론 과정을 거친단 말임

그래서 이걸 모방해서 모델에게 풀이 과정까지 입력하거나 “think step by step” 이런걸 추가해 모델이 알아서 단계적 추론을 하도록 유도하는 기법이 Chain of Thought임